

Replication Study of a Portfolio Optimization Model Using LSTM and Random Forest Claims

Hasan Berkan Eroğlu
M.Sc. Student, Ankara Yıldırım Beyazıt University
CENG524 – Advanced Machine Learning
Email: 245101115@aybu.edu.tr

Abstract—This report presents a replication study of the paper *A Comparative Analysis of Portfolio Optimization: LSTM vs. Random Forests* by Jinyuan Sun. The original paper studies portfolio optimization using four healthcare and insurance-related stocks, namely LLY, PGR, CI, and UNH, over the period from May 10, 2019 to October 16, 2023. The paper claims to compare Long Short-Term Memory (LSTM) and Random Forest models with a NASDAQ benchmark using a mean-variance portfolio optimization strategy.

The replication used the publicly released project repository and exact CSV artifacts distributed with the implementation. Before execution, the dataset files were inspected and fingerprinted. The repository contained 100 readable stock-level CSV files under the `archive` directory, a readable `nasdaq31.csv` file, and one empty `sp500.csv` file. The released notebook was executed first without methodological modification. The result showed that the benchmark return was stable, but the model-based portfolio returns changed across repeated executions because random seeds were not fixed. A controlled version was then produced by fixing NumPy, Random Forest, Decision Tree, and Monte Carlo random states. This made the implementation deterministic.

The main finding is that the official implementation does not reproduce the paper’s stated methodology. The notebook trains Random Forest and Decision Tree models, not an LSTM model. Therefore, the central LSTM-versus-Random-Forest claim cannot be verified from the released implementation.

Index Terms—Portfolio optimization, replication study, LSTM, Random Forest, Decision Tree, mean-variance optimization, reproducibility, fintech.

I. INTRODUCTION

Portfolio optimization is an important problem in financial technology. The main goal is to allocate capital across different assets in a way that improves expected return while controlling risk. Classical approaches such as mean-variance optimization are still widely used, but recent studies increasingly combine them with machine learning models for return forecasting.

The aim of this project is to replicate the paper *A Comparative Analysis of Portfolio Optimization: LSTM vs. Random Forests* [1]. The original paper claims that LSTM and Random Forest models are trained on stock price data and that the resulting portfolio returns are compared with a NASDAQ benchmark. This replication does not attempt to design a new portfolio model. Its purpose is to verify whether the reported experimental pipeline can be reproduced using the publicly available data and implementation artifacts.

The replication follows two stages. First, the official notebook is executed as released, with no methodological change.

Second, a controlled version is executed with fixed random seeds to test whether the observed numerical instability is caused by uncontrolled randomness. The final objective is to identify which parts of the original study are reproducible and which parts are not.

II. ORIGINAL PAPER SUMMARY

The original paper studies a portfolio optimization problem using four selected stocks: LLY, PGR, CI, and UNH. These stocks are described as healthcare and insurance-related assets. The paper states that historical stock data from May 10, 2019 to October 16, 2023 are used. It also states that the first 1117 market days are used for model training and the final 30 market days are used for portfolio evaluation [1].

The paper describes two prediction methods. The first is an LSTM model. The stated LSTM architecture uses TensorFlow Sequential API, two LSTM layers, and dense layers. The second method is Random Forest regression. The predicted stock prices are converted into returns. These returns are then used in a mean-variance portfolio optimization framework. The final portfolio performance is compared with a NASDAQ benchmark.

The reported conclusion is that both model-based portfolios outperform the NASDAQ benchmark and that the LSTM model performs better than Random Forest. Because this is the central claim of the paper, the replication focused on whether the released implementation actually supports this claim.

III. RELATED WORK

Mean-variance portfolio optimization originates from Markowitz’s modern portfolio theory [3]. In this framework, asset allocation is treated as a trade-off between expected return and portfolio variance. More recent studies combine machine learning prediction models with portfolio optimization. For example, machine learning models can be used to forecast future returns, and the predicted returns can then be inserted into a mean-variance allocation process [4], [5].

The paper selected for this replication belongs to this machine-learning-assisted portfolio optimization family. Its stated contribution is not a new optimization theory, but a practical comparison between LSTM and Random Forest prediction models in a portfolio construction setting. This makes the paper suitable for a replication study because the expected

output is concrete: dataset loading, model training, predicted returns, portfolio returns, and benchmark comparison.

IV. DATA AVAILABILITY AND DATASET DESCRIPTION

The original paper states that the daily stock data were obtained from the Kaggle dataset *Top 100 NYSE daily stock prices* [2]. The dataset link reported in the paper and used in this replication is:

<https://www.kaggle.com/datasets/svaningelgem/nyse-100-daily-stock-prices/data>

For exact-data replication, the study did not redownload or reconstruct data from a live market API. Instead, the publicly released repository snapshot was used. This is important because financial data can change due to corrections, corporate actions, adjusted-price recalculation, or vendor-specific revisions. Using the repository’s fixed CSV files provides a more stable basis for replication than downloading data again.

The local repository contained 102 CSV files. Among these, 100 stock-level CSV files were available under the `archive` directory. The file `nasdaq31.csv` was readable and was used by the notebook as the NASDAQ benchmark. The file `sp500.csv` was present but empty. Since the notebook did not use `sp500.csv`, this was treated as a repository quality issue rather than a direct execution blocker.

The stock-level CSV files had the following schema:

`ticker, date, open, high, low, close`

A major issue was found here. The paper states that “Adj Close” is used as the daily price. However, the released stock CSV files do not contain an adjusted close column. The implementation uses the raw `close` column. This creates a methodological mismatch between the paper description and the actual released implementation.

V. METHODOLOGY

The replication methodology had three main parts: dataset verification, as-released execution, and controlled execution.

A. Dataset Verification

Before running the notebook, all CSV files were inspected. The verification recorded file size, readability, number of rows, number of columns, and column names. This was done to confirm whether the repository contains the data required for the experiment. The dataset check confirmed that all 100 stock-level files were readable and consistently structured. It also confirmed that `sp500.csv` was empty.

B. As-Released Execution

The notebook `app.ipynb` was copied and executed as `app_execution_as_released.ipynb`. The goal was to test whether the official implementation reproduces its own outputs without modification. The notebook was executed from a fresh Jupyter kernel. After that, the notebook was rerun several times from fresh kernels to test numerical stability.

C. Controlled Execution

After the as-released executions showed unstable model-based returns, a controlled version was created as `app_execution_as_released_controlled.ipynb`. The same dataset, same stock selection, and same portfolio logic were preserved. Only random seeds were fixed. The following changes were applied:

- `np.random.seed(42)` was added.
- `random_state=42` was added to `RandomForestRegressor`.
- `random_state=42` was added to `DecisionTreeRegressor`.
- The NumPy random seed was reset before Monte Carlo portfolio simulation.

These changes were not used to repair the scientific method of the paper. They were used only to test whether the released implementation could become deterministic when randomness was controlled.

VI. EXPERIMENTAL SETUP

The experiments were executed locally using Python and Jupyter Notebook. The repository was kept in a local project folder. The notebook was executed with the repository root as the working directory because it uses relative paths such as `archive` and `nasdaq31.csv`.

The released notebook creates a matrix of 100 stocks, removes rows with missing values, and then splits the data into a known part and an unknown part. The replicated execution produced the following shapes:

TABLE I
DATASET SPLIT PRODUCED BY THE RELEASED NOTEBOOK

Dataset part	Rows	Columns
Total after <code>dropna()</code>	1117	100
Known/training part	1087	100
Unknown/test part	30	100

This does not match the paper’s statement that the study uses 1147 total market days and 1117 training days. The implementation uses 1117 total rows after missing-value removal, not 1117 training rows.

VII. RESULTS

A. As-Released Replication Results

The as-released notebook was executed multiple times. The NASDAQ return was stable in every execution. However, the model-based portfolio returns changed between fresh-kernel runs.

The results show that the benchmark calculation is deterministic, but the portfolio model outputs are not. This is mainly because the notebook does not set random seeds for the machine learning models or for Monte Carlo portfolio sampling.

TABLE II
REPEATED EXECUTIONS OF THE RELEASED NOTEBOOK

Execution	NASDAQ return	RF return	DT return
Saved output in released notebook	-0.0344847098	0.1581967765	0.1072814419
Fresh-kernel run 1	-0.0344847098	0.1579421955	0.1727922072
Fresh-kernel run 2	-0.0344847098	0.1508853359	0.1142518220
Fresh-kernel run 3	-0.0344847098	0.1474317305	0.1345048082
Fresh-kernel run 4	-0.0344847098	0.1470093299	0.1296921705

B. Controlled Replication Results

The controlled notebook was executed three times from fresh kernels. The results were identical across all runs.

TABLE III
CONTROLLED DETERMINISTIC EXECUTION RESULTS

Run	NASDAQ	RF	DT
Controlled run 1	-0.0344847098	0.1492175467	0.1039672433
Controlled run 2	-0.0344847098	0.1492175467	0.1039672433
Controlled run 3	-0.0344847098	0.1492175467	0.1039672433

This confirms that the earlier variability was caused by uncontrolled randomness. After random states were fixed, the implemented experiment became reproducible. However, the controlled implementation still does not reproduce the paper’s claimed LSTM experiment because the notebook contains no LSTM model.

VIII. MAIN REPRODUCIBILITY FINDINGS

The most important findings are summarized in Table IV.

The most serious issue is the absence of the LSTM model in the official implementation. This means the central claim of the paper cannot be verified from the released code. The implementation can be made deterministic, but only for Random Forest and Decision Tree models.

IX. DISCUSSION

The replication produced a clear outcome. The public data artifacts and official notebook were sufficient to execute a portfolio experiment. However, the executed experiment is not the same as the experiment described in the paper.

The paper presents the study as a comparison between LSTM and Random Forest. The released notebook imports and trains Random Forest and Decision Tree regressors. It does not import TensorFlow, Keras, PyTorch, or any LSTM implementation. Later, the notebook labels the Random Forest portfolio plot as “LSTM” and the Decision Tree portfolio plot as “Random Forest”. This creates a direct mismatch between the paper’s terminology and the executable code.

Another important problem is the data column mismatch. The paper says that adjusted close prices are used. The released dataset contains only raw close prices. In financial studies, this distinction matters because adjusted prices account for dividends, splits, and other corporate actions. Using raw close prices can change returns and therefore portfolio weights.

The uncontrolled executions also show a reproducibility problem. The same notebook and same data produced different model-based portfolio returns across fresh-kernel runs. This does not necessarily indicate fabrication by itself. It shows that the implementation is not deterministic. However, combined with the missing LSTM implementation and mislabeled outputs, it prevents verification of the paper’s main claim.

X. LIMITATIONS

This replication has some limitations. First, it relies on the public repository snapshot available during the project. If the author updates the repository later, the available implementation may change. Second, the replication did not attempt to reconstruct a new LSTM model independently. This decision was intentional because the goal was to verify the paper using the released implementation, not to replace missing code with a new implementation.

Third, the controlled version fixes random seeds but does not correct all methodological issues. For example, it does not replace raw close prices with adjusted close prices, it does not implement covariance shrinkage, and it does not fix the hard-coded day index in the portfolio simulation. These issues were preserved because the controlled run was meant to diagnose determinism, not to redesign the study.

Finally, the original paper does not provide detailed numerical tables for all intermediate outputs. Much of the comparison therefore depends on the released notebook outputs and the visual claims in the paper.

XI. CONCLUSION

This project attempted to replicate the paper *A Comparative Analysis of Portfolio Optimization: LSTM vs. Random Forests*. The exact public data artifacts and official implementation were used. The dataset package was mostly usable: 100 stock-level CSV files and the NASDAQ benchmark file were readable, while `sp500.csv` was empty but unused.

The as-released notebook could be executed, but repeated fresh-kernel runs produced unstable model-based portfolio returns due to uncontrolled randomness. After adding fixed random seeds, the results became deterministic: NASDAQ return was -0.0344847098, Random Forest portfolio return was 0.1492175467, and Decision Tree portfolio return was 0.1039672433.

The main conclusion is that the paper’s central claim could not be verified. The released implementation does not train an LSTM model. It trains Random Forest and Decision Tree models, while the resulting plots are labeled as LSTM and Random

TABLE IV
COMPARISON BETWEEN PAPER CLAIMS AND RELEASED IMPLEMENTATION

Aspect	Paper description	Released implementation finding
Main comparison LSTM model	LSTM versus Random Forest TensorFlow Sequential API with LSTM layers and dense layers	Random Forest versus Decision Tree No TensorFlow/Keras LSTM implementation found in the notebook
Selected assets Price variable	LLY, PGR, CI, UNH Adjusted close price	Same four selected assets are used Raw <code>close</code> column is used; no adjusted close column exists in CSV files
Dataset split	1147 total days, 1117 training days, 30 test days	1117 total rows after <code>dropna()</code> , 1087 known rows, 30 unknown rows
Covariance method	Shrinkage covariance	Ordinary sample covariance through <code>returns.cov()</code>
Randomness control	Not clearly specified	No seeds in released notebook; repeated runs produce different returns
Model labeling	Results described as LSTM and Random Forest	RF result is plotted with title “LSTM” and DT result is plotted with title “Random Forest”
Portfolio day index	Daily portfolio optimization implied	Portfolio simulation uses a hard-coded day index in the return-generation call
Repository quality	Complete reproducibility package expected	<code>sp500.csv</code> exists but is empty; final stale notebook cell references undefined variable <code>na</code>

Forest. Additional mismatches were also found, including raw close prices instead of adjusted close prices, sample covariance instead of shrinkage covariance, and a different training/test split from the one stated in the paper.

Therefore, this replication does not support the paper’s stated LSTM-versus-Random-Forest conclusion. The released code supports only a Random-Forest-versus-Decision-Tree experiment, and even that experiment required deterministic patching to become exactly reproducible.

DATA AND CODE AVAILABILITY

The original data source reported by the paper is the Kaggle dataset *Top 100 NYSE daily stock prices*: <https://www.kaggle.com/datasets/svaningelgem/nyse-100-daily-stock-prices/data>. The replication used the fixed CSV files distributed in the author’s public project repository [7] instead of redownloading data from Kaggle.

The replication artifacts produced for this study, including the as-released execution notebook, the controlled deterministic execution notebook, the dataset manifest, and the LaTeX report source, are made available in the author’s replication repository [8].

REFERENCES

- [1] J. Sun, “A Comparative Analysis of Portfolio Optimization: LSTM vs. Random Forests,” 2024.
- [2] S. Van Ingelgem, “Top 100 NYSE daily stock prices,” Kaggle dataset. Available: <https://www.kaggle.com/datasets/svaningelgem/nyse-100-daily-stock-prices/data>
- [3] H. Markowitz, “Portfolio Selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [4] A. Chaweeewanchon and R. Chaysiri, “Markowitz Mean-Variance Portfolio Optimization with Predictive Stock Selection Using Machine Learning,” *International Journal of Financial Studies*, vol. 10, no. 3, 2022.
- [5] W. Chen, H. Zhang, M. K. Mehlawat, and L. Jia, “Mean–variance portfolio optimization using machine learning-based stock price prediction,” *Applied Soft Computing*, vol. 100, 2021.
- [6] T. Fischer and C. Krauss, “Deep learning with long short-term memory networks for financial market predictions,” *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, 2018.

- [7] J. Sun, “portfolio_optimization: A Comparative Analysis of Portfolio Optimization: LSTM vs. Random Forests,” GitHub repository. Available: https://github.com/Jinyuan-S/portfolio_optimization
- [8] H. B. Eroglu, “paper_replication_study_524: CENG524 Portfolio Optimization Replication Study,” GitHub repository. Available: https://github.com/HBE18/paper_replication_study_524